

Fine-tuning, Reinforcement Learning and Parameter-Efficient Adaptation

Dimitris Tsirmpas

Athens University of Economics and Business

Archimedes, Athena Research Center

June 27, 2026

What we will be talking about

- 1 Naively Finetuning LLMs
 - What is finetuning?
 - Why not (fully) finetune LLMs?
- 2 Making finetuning practical through LoRA
 - Understanding why LoRA works
 - How to use LoRA
- 3 Making finetuning scalable through QLoRA
 - Quantizing LLMs: Why and how?
 - How does it differ from LoRA?

Outline

1 Naively Finetuning LLMs

- What is finetuning?
- Why not (fully) finetune LLMs?

2 Making finetuning practical through LoRA

- Understanding why LoRA works
- How to use LoRA

3 Making finetuning scalable through QLoRA

- Quantizing LLMs: Why and how?
- How does it differ from LoRA?

Section 1

Naively Finetuning LLMs

Subsection 1

What is finetuning?

Why finetune LLMs?

- **Prompting:** Changes **input behavior**. We guide the model with instructions and examples.
- **Fine-Tuning:** Changes **model parameters**. We switch the **knowledge** of the model.

Reminder

Fine-Tuning (FT): The process of further training a pre-trained model on a smaller, domain-specific dataset to adapt its general knowledge to a specialized task.

Pretraining vs Finetuning

Reminder

- **Pre-training** provides the model with initial, **general** knowledge by utilizing **any** and all data sources.
- **Finetuning** shifts existing knowledge, to **specific** domains using **domain-specific** datasets.

Example: Domain-specific tasks

Part-of-Speech (POS) Tagging

The process of assigning grammatical categories to each word in a sentence.

POS Tagging

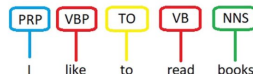


Figure:

<https://byteiota.com/pos-tagging/>

Example: Domain-specific tasks

Biomedical POS Tagging

- Patterns absent or rare in general-domain text.
- Prompting probably not enough due to **missing knowledge**.

Word	POS Tag
Homozygous	JJ
null	JJ
mutants	NNS
die	VBP
shortly	IN
after	RB
birth	NN

Table: Example from the CRAFT dataset.

Section 1

Naively Finetuning LLMs

Subsection 2

Why not (fully) finetune LLMs?

High Resource Cost

- **Computational Power:** Full FT requires **massive hardware requirements** and **thousands of compute hours** for large models (e.g., Llama-70B).
- **Storage:** Each specialized task requires a separate, full copy of the massive foundational model.

Operational Overhead

- Training is **slow**.
- Deployment becomes complex, managing **multiple large models** for different applications.
 - A single 13B model may require 26GB of storage.
 - 10 finetuned models may thus require 260GB.
- High risk of **Catastrophic Forgetting**—the model forgets its general knowledge while learning the specific task.

Scalability issues

BERT

- **Model Size:** ~340M Parameters
- **VRAM Requirement:** $\approx$ 8-12GB
- **Training Time:** Minutes to a few hours

Llama 70B

- **Model Size:** ~70 Billion Parameters
- **VRAM Requirement:** $\approx$ 1000GB+
- **Training Time:** Days to Weeks (even with specialized hardware)

LLMs are **too large** to run full-finetuning. But do we need to?

Parameter-Efficient Fine-Tuning (PEFT)

Definition

PEFT: A set of techniques used to adapt large pre-trained LLMs to specific tasks by updating only a **small subset of their parameters**, rather than retraining the entire model.

- Many algorithms developed through the years (adapters, prompt/prefix tuning etc.).
- However, the **LoRA** family of methods has dominated the LLM space in the last few years.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Outline

- 1 Naively Finetuning LLMs
 - What is finetuning?
 - Why not (fully) finetune LLMs?
- 2 Making finetuning practical through LoRA
 - Understanding why LoRA works
 - How to use LoRA
- 3 Making finetuning scalable through QLoRA
 - Quantizing LLMs: Why and how?
 - How does it differ from LoRA?

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Section 2

Making finetuning practical through LoRA

Subsection 1

Understanding why LoRA works

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

PEFT: LoRA

- Low-Rank (LoRA) Matrix Decomposition is a specific PEFT algorithm that exploits the fact that weight updates in LLMs have a “low intrinsic rank”.
- Low rank matrices enable us to scalably represent information.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Low-rank matrices

What is a matrix?

A matrix is essentially a organized collection of data (rows and columns). For large datasets, these matrices can become incredibly complex, requiring massive storage and computational power.

Example: Data matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix}$$

Low-rank matrices

Matrix rank

- Rank r represents the number of genuinely independent columns (or rows!) within the matrix M .
- If $M \in \mathbb{R}^{m \times n}$, then $\text{rank}(M) \leq \min(m, n)$.

Example: Music

- Imagine a piano with 88 keys.
- Most songs don't require all 88 keys; they can be described by melody, rhythm etc.
- Low-rank updates capture changing **general patterns**.

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Low-rank matrices

Rank Factorization

If a matrix M has a low rank r , we can approximate it (or exactly represent it) by factoring it into a product of two simpler matrices.

Example: Rank 1 matrix

$$\begin{bmatrix} 2 & 4 & 6 & 8 & 10 \\ 1 & 2 & 3 & 4 & 5 \\ 3 & 6 & 9 & 12 & 15 \\ 4 & 8 & 12 & 16 & 20 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 3 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \end{bmatrix}.$$

- └ Making finetuning practical through LoRA
- └ Understanding why LoRA works

Why are LLM updates typically low-rank?

Many possible explanations

- Updates generally follow the direction of existing knowledge → the updates may often just be **re-weighting** of existing weights.
- Underlying **ML problems** are usually inherently low-rank.

Why are LLM updates typically low-rank?

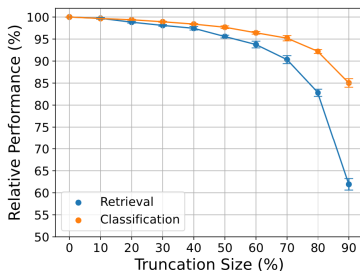


Figure: <https://aclanthology.org/2025.emnlp-main.1410.pdf>

Large embedding spaces

Modern NLP models are **over-parameterized** → not enough information to fill in higher ranks of weights.

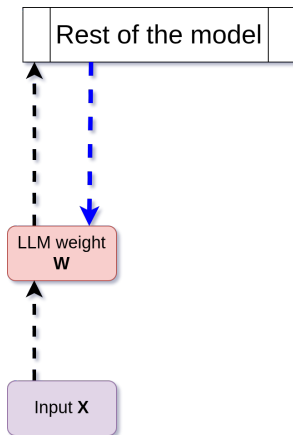
LoRA: Low-rank decomposition

$$\underbrace{A_{m \times n}}_{\text{large matrix}} \approx \underbrace{U_{m \times r}}_{\text{left factors}} \underbrace{V_{r \times n}}_{\text{right factors}}, \quad r \ll \min(m, n).$$

$$A_{10000 \times 5000} \approx U_{10000 \times 50} V_{50 \times 5000},$$

$$\underbrace{10000 \times 5000}_{50,000,000 \text{ parameters}} \longrightarrow \underbrace{10000 \times 50 + 50 \times 5000}_{750,000 \text{ parameters}}.$$

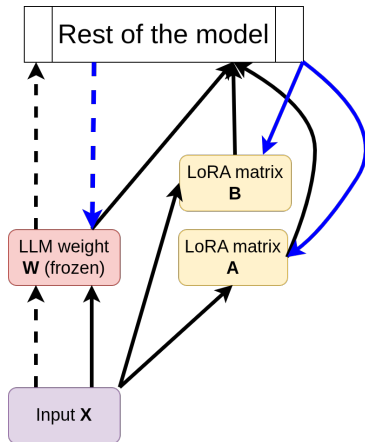
LoRA: Mathematical Foundation



Full SFT

- $W \in \mathbb{R}^{d \times k}$ weights.
- X input, h output.
- Fwd pass: $h = WX$.
- Bckwd pass:
 $W = W + \alpha \nabla h$

LoRA: Mathematical Foundation



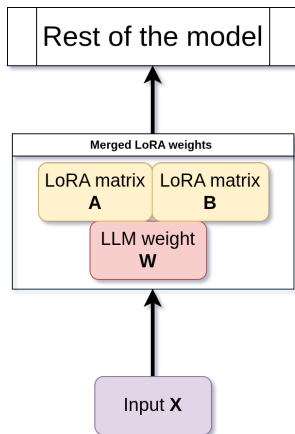
LoRA

- $A \in \mathbb{R}^{r \times k} \rightarrow$ Gaussian.
- $B \in \mathbb{R}^{d \times r} \rightarrow$ zeros.
- $r \ll \min(d, k)$, hence A, B are much smaller than W .

Example: Music

A could represent the genre of music, while B the melody, rhythm etc.

LoRA: Mathematical Foundation



LoRA

- During inference, we compute
$$H = WX + \Delta WX.$$
- Alternatively, merge weights:
$$W' = W + \frac{\alpha}{r}(AB)$$

LoRA: Algorithm

▷ =====*Initialization*===== ◁

for all $W \in all_weights$ **do**

└ Set $\mathbf{W}$ to read-only ▷ *Original knowledge pre-served*

for all $W_u \in target_modules$ **do**

Initialize $\mathbf{A}$ randomly

Initialize $\mathbf{B} = \mathbf{0}$ ▷ *Contributes zero to the output initially*

▷ =====*Backpropagation*===== ◁

for all $X \in input_embeds$ **do**

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$$

$$\mathbf{H} = \mathbf{W} \mathbf{X} + \Delta \mathbf{W} \mathbf{X} \quad \triangleright \text{Output} = \text{original} + \text{adaptation}$$

▷ =====*End of training*===== ◁

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r} (\mathbf{A} \mathbf{B}) \quad \triangleright \text{Merge original and adaptation}$$

Section 2

Making finetuning practical through LoRA

Subsection 2

How to use LoRA

LoRA: Practical Considerations

■ How many ranks do I use?

- $r \in [1, 16]$: fast, easy, recommended for minor adjustments.
- $r \in [16, 64]$: recommended for most domain adaptation scenarios.
- $r \in [64, 256]$: may hit computational bottlenecks, only use for dramatic shifts in domain and knowledge.

■ How do I set α ?

- Usually set to a multiple of r .
- Very low $\alpha \rightarrow$ minimal contribution. Very high $\alpha \rightarrow$ destruction of model knowledge.
- $\alpha = 2r$ usually used as a starting point.

LoRA: Practical Considerations

- What modules do I change?
- Start from the top of this list and go down as required:
 - 1 **Query (q_proj) and Value (v_proj):** Most frequently adapted; control **how the model attends** to input.
 - 2 **Key (k_proj):** Increases **information retrieval** capacity.
 - 3 **Output (o_proj):** Less common, but useful for modifying **integrated** attended information.
 - 4 **Feed-forward Networks (MLP):** Possible to adapt, but generally offer less parameter benefit compared to attention layers.

LoRA: Practical Considerations

- Should we merge the LoRA weights?

$$\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}(\mathbf{AB})$$

Yes

- Lowers **GPU memory** and execution **time**.
 - More significant if r is large.
- Deployment may be easier.

No

- You can use the same model with **many** finetuned adapters which are **dynamically selected** depending on your task.

- └ Making finetuning scalable through QLoRA
- └ Quantizing LLMs: Why and how?

Outline

- 1 Naively Finetuning LLMs
 - What is finetuning?
 - Why not (fully) finetune LLMs?
- 2 Making finetuning practical through LoRA
 - Understanding why LoRA works
 - How to use LoRA
- 3 Making finetuning scalable through QLoRA
 - Quantizing LLMs: Why and how?
 - How does it differ from LoRA?

- └ Making finetuning scalable through QLoRA
- └ Quantizing LLMs: Why and how?

Section 3

Making finetuning scalable through QLoRA

Subsection 1

Quantizing LLMs: Why and how?

QLoRA: Quantization

- Quantization refers to dropping the size (not number) of the LLM parameters.
- Normally, each parameter is stored in a 16 bit floating point.
- Quantization can drop them to usually 4 bits, resulting in **×4 less** GPU memory and inference costs.



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

- └ Making finetuning scalable through QLoRA
- └ Quantizing LLMs: Why and how?

QLoRA: Quantization

- Quantization is one of the **most useful tools** at your disposal.
- Allows for training and inference in consumer-grade hardware.
- We will return to this in Module 5.



Figure: <https://developer.nvidia.com/blog/model-quantization-concepts-methods-and-why-it-matters>

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

Section 3

Making finetuning scalable through QLoRA

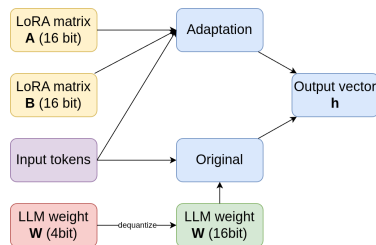
Subsection 2

How does it differ from LoRA?

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

QLoRA: Concept

- Load a 4-bit quantized model using a normal approximation of weights.
- Run LoRA in full precision, **dequantizing** the model weights **one at a time**.
- Continue as normal.



- └ Making finetuning scalable through QLoRA

- └ How does it differ from LoRA?

QLoRA: Algorithm

▷ =====*Initialization*=====

▷ *Same as before*

▷ =====*Backpropagation*=====

for all $X \in \text{input_embeds}$ **do**

$$\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{A} \mathbf{B}$$

$\mathbf{H} = \text{dequant}(\mathbf{W}) X + \Delta \mathbf{W} X$ ▷ *Output = original + adaptation*

▷ =====*End of training*=====

▷ *Only attempt dynamic merge. Ignore this advice at your own peril, or if you really know what you are doing.*

└ Making finetuning scalable through QLoRA

└ How does it differ from LoRA?

QLoRA: Pros and cons

Pros

- Frozen weights no longer monopolize **GPU memory** and inference **time**.
- Allows training in **consumer-grade** hardware.
- Massively **reduces training** times.

Cons

- End-result is a quantized → **less powerful** model.
- Dequantization imposes a $\approx 10\%$ tax on training times.
- Merging is now much messier.

- └ Making finetuning scalable through QLoRA
- └ How does it differ from LoRA?

Summing up

- Full finetuning for LLMs is out of the question.
- However, weight updates are usually **low-rank**.
- Thus, we can **approximate** full finetuning with LoRA.
- We may use QLoRA instead to make it much more **scalable**.